

Signaux et Systèmes 2/Signals and Systems 2

Stéphane Rossignol – stephane.rossignol@centralesupelec.fr

September 14, 2026

Here are these slides

Teams – > SIG2-PHYS – > Fichiers – > Supports de cours – >

Be aware: this version should evolve

Plan

- ▶ **Part 1: Introduction**
- ▶ Part 2: More about standard (non-parametric) spectral analysis (periodogram...)
- ▶ Part 3: Parametric spectral analysis: AR/MA/ARMA
- ▶ Part 4: Advanced spectral analysis: MUSIC, Pisarenko, Prony
- ▶ Part 5: Laplace, Z, Bode...
- ▶ Part 6: Filtering: analog versus digital
- ▶ Part 7: Advanced spectral analysis: Wavelets
- ▶ Part 8: Matching/basis pursuit
- ▶ Part 9: Sparsity
- ▶ Part 10: Random signals: temporal/spectral properties

My works are linked to these lessons

- ▶ Studies at the university: DUT EEEII, “licence” and “maîtrise” (M1) in electronics, DEA (M2) “signal télécom image radar (option radar)”, DEA (M2) “instrumentation en sciences de l’univers”
- ▶ Military service, as “scientifique du contingent” – detection of 300 tonnes ferromagnetic masses, moving under water
- ▶ PhD-Thesis (ended in July 2000); on the segmentation and indexing of musical signals; done at IRCAM and here, with France Télécom-Orange as partner
- ▶ Modification of the prosody of an actor (Depardieu speaking english, in Vatel from Roland Joffé)
- ▶ Postdoc (2000-2001): modelisation of the vibrato, in a Dutch university (⇒ composition assistance, MPEG-7)
- ▶ Postdoc (2001-2005): implementation of dialog systems, in a Dutch university (european project COMIC)

My works are linked to these lessons

- ▶ Postdoc (2006): implementation of an Automatic Speech Recognizer (for France Télécom-Orange)
- ▶ Postdoc (2007): characterization of loud-speakers defects on the production line (automotive supplier)
- ▶ Assistant-professor, here, since 2008
 - ▶ my studies: music processing (and also speech processing)
 - ▶ implementation of dialog systems (european projects CLASSIC, ALLEGRO, HILAIRE)
 - ▶ holophony
 - ▶ teaching, mostly in signal processing (SIG1, SIG2, RASS, spectral analysis, sound processing)
- ▶ Multispeech TEAM, LORIA (in Nancy)
- ▶ Predictive maintenance (with C. Tanougast)

Where am I to find?

- ▶ third floor, LORIA wing
- ▶ office: B306
- ▶ phone: 01 75 31 70 74
- ▶ email: stephane.rossignol@centralesupelec.fr

Organization

- ▶ Lessons (Cours Magistraux):
 - ▶ there is a syllabus ; I don't know if I will have time to do everything
- ▶ Lab works (TL/TD):
 - ▶ I give some code in Octave/Matlab/Python/etc.; choose the language you prefer
 - ▶ work in pair; I will create a Teams canal for each pair
- ▶ Exam:
 - ▶ during the lab works, write a report describing what you do; this is what I will evaluate at the end

What are the goals of these lessons?

- ▶ First, the uncertainty principle of Gabor is very annoying; is there a way to disregard it?
 - ▶ Obviously **no**...
 - ▶ HOWEVER: a sine is characterized not only by its frequency, but also by its amplitude and its phase
 - ▶ Parametric spectral analysis methods (AR/MA/ARMA) and other advanced spectral analysis methods (MUSIC, Pisarenko, Prony), try to trick the uncertainty principle of Gabor taking into account this consideration (Wavelets, to a certain extent, do the same)
 - ▶ Note: those spectral analysis methods are called “high-resolution spectral analysis methods”, for this reason

What are the goals of these lessons?

- ▶ Second, is there a way to disregard the “ $3T_0$ rule”, which strongly limits the performance of the Fourier analysis, in terms of temporal resolution?
 - ▶ It is not obvious... but, fortunately, **yes**...
 - ▶ The Fourier transform cannot adapt T_0 to the frequency band it analyses, due to strong algorithmic reasons (the FFT algorithm)
 - ▶ BUT: Advanced spectral analysis like Wavelets take full account of this consideration $\Rightarrow$ therefore, I will have to give more insight about how analog and digital Filters are built, and then to speak again about Laplace transform, Z transform, Bode representations

What are the goals of these lessons?

- ▶ Third, what about processing signals that are more “complicated” than harmonic (say, sound) signals, and can we go further?
 - ▶ what about sparsity?
 - ▶ why choose a method of analysis, between non-parametric/parametric/wavelets/etc. methods: is it possible to mix all these analyses and find a way to automatically find out the best? The “pursuit” schemes try to do that.
 - ▶ random signals: their temporal/spectral properties and the obtained spectral analyses

Plan

- ▶ Part 1: Introduction
- ▶ **Part 2: More about standard (non-parametric) spectral analysis (periodogram...)**
- ▶ Part 3: Parametric spectral analysis: AR/MA/ARMA
- ▶ Part 4: Advanced spectral analysis: MUSIC, Pisarenko, Prony
- ▶ Part 5: Laplace, Z, Bode...
- ▶ Part 6: Filtering: analog versus digital
- ▶ Part 7: Advanced spectral analysis: Wavelets
- ▶ Part 8: Matching/basis pursuit
- ▶ Part 9: Sparsity
- ▶ Part 10: Random signals: temporal/spectral properties

Part 2 – Non-parametric spectral analysis

Stéphane Rossignol

September 14, 2026

Plan

- ▶ Periodogram
- ▶ Windowing
- ▶ Daniell estimator
- ▶ Bartlett estimator
- ▶ Welch estimator
- ▶ Correlogram (Blackman-Tukey)

The periodogram (1898) (squared module of the DFT, computed in practice with the FFT: 1965), is, for a given frequency:

- ▶ a biased estimator of the real PSD: but fortunately the bias tends towards 0 when N (the number of samples in the processed frame) tends to infinity
- ▶ and a non-consistent estimator: its variance does not tend to 0 when N tends to infinity (thus, abandonment of the periodogram until 1950)
 - ▶ on the other hand, for a Gaussian white noise, the correlation of the periodogram for 2 distinct frequencies is very quickly small as soon as the 2 frequencies move away (and even for N small): so, when N increases, the periodogram remains very rowdy (“chahuté”)

⇒ goal: to minimize (in particular) the variance of the periodogram; we end up with a compromise between the variance of the estimator and its resolution

Periodogram:

$$\blacktriangleright \hat{S}(\nu) = \frac{1}{N} |X(\nu)|^2$$

Bias:

$$\blacktriangleright E[\hat{S}(\nu_1)] = W_B(\nu_1) * S(\nu_1) \text{ where } W_B(\nu) \text{ is the FT of the triangular window, of width } N \text{ samples}$$

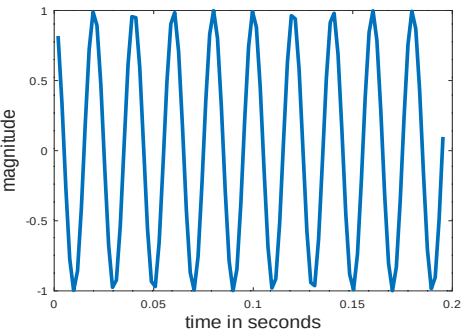
Variance (for a gaussian white noise):

$$\blacktriangleright \text{var}[\hat{S}(\nu_1)] = S(\nu_1)^2 \left(1 + \frac{\sin(2\pi\nu_1 N)}{N \sin(2\pi\nu_1)}\right)^2$$

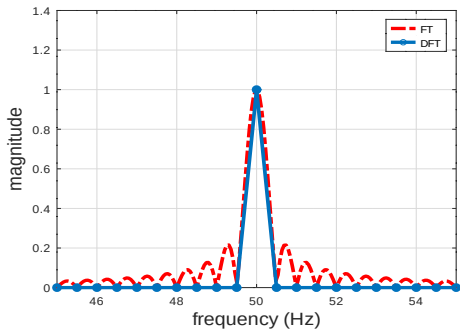
$\Rightarrow$ see periodogre.m

Windowing – Effect of frequency sampling (1/3)

signal frame: sine wave of frequency 50 Hz



$f_e=512$ Hz, $f_0=50$ Hz, $N_{sig}=1024$, no 0-padding

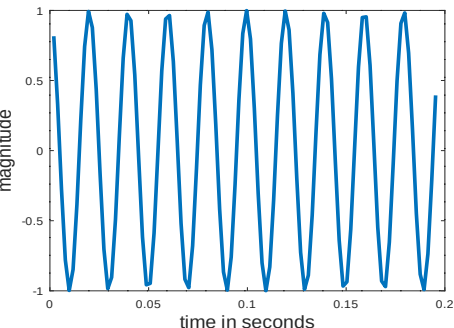


(note: magnitude multiplied by 2)

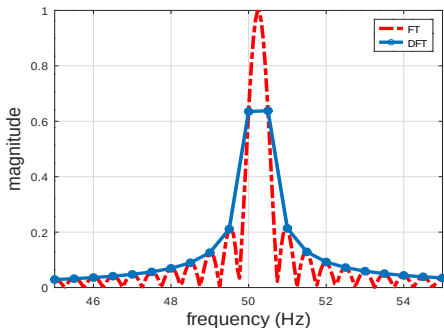
If $\nu_0 = k/N$, ν_0 is a sampled frequency of the DFT

Effect of frequency sampling (2/3)

signal frame: sine wave of frequency 50.25 Hz



fe=512 Hz, f0=50.25 Hz, Nsig=1024, no 0-padding



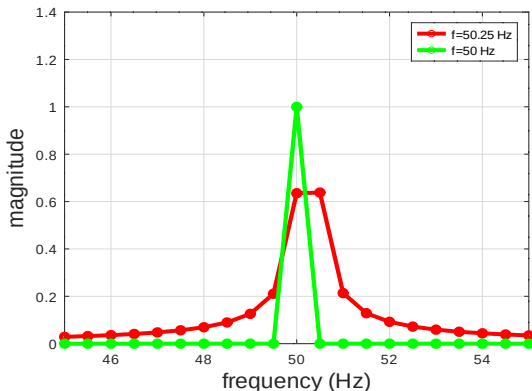
If $\nu_0 \neq k/N$, uncertainty proportionnal to $1/N$ on the frequency,
and uncertainty on the magnitude

Note: the 0-padding and the windowing help to solve this

Effect of frequency sampling (3/3)

Comparison between the 2 previous spectra:

fe=512 Hz, Nsig=1024, no 0-padding



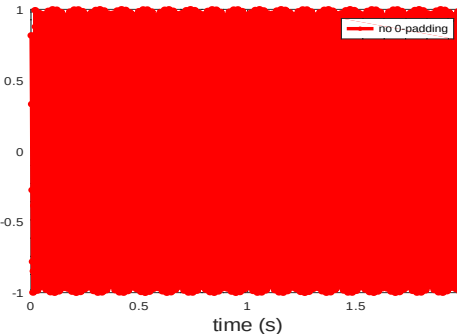
Windowing – Effect of the 0-padding (1/4)

- ▶ Definition: this means lengthening a signal of length P useful samples by adding M zeros at the end, to obtain a signal of size $N = P + M$
- ▶ Improvement of the **visualization/representation** of the results
 - ▶ $\Rightarrow$ **attention**: no more useful signal samples are taken, so the zero-padding (“bourrage de zéros”, in French) does not provide any additional information (simple interpolation of the points of the FFT of length P)
 - ▶ in fact, we sample the FT with a step of $1/N$, and not any more with a step of $1/P$
- ▶ Also used (and this is important!) to be in the configuration $N = 2^r$, with r integer

Effect of the 0-padding (2/4) – see `exemples_nplusi.m`

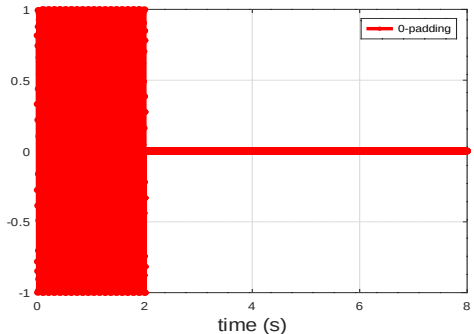
Signal without 0-padding:

$f_e=512$ Hz, $N_{sig}=1024$, $f_0=50.25$ Hz



Signal with 0-padding:

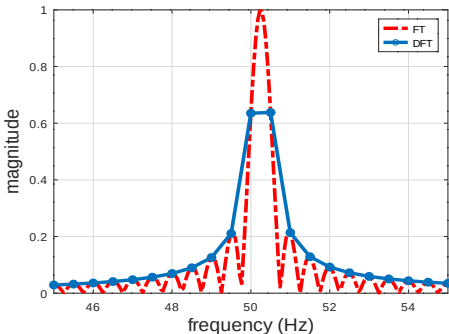
$f_e=512$ Hz, $N_{sig}=1024$, $N_{fft}=4096$, $f_0=50.25$ Hz



Effect of the 0-padding (3/4) – see `exemples_nplusi.m`

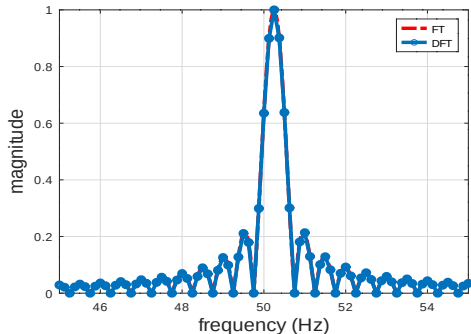
FFT signal without 0-padding

$f_e=512$ Hz, $f_0=50.25$ Hz, $N_{\text{sig}}=1024$, no 0-padding



FFT signal with 0-padding

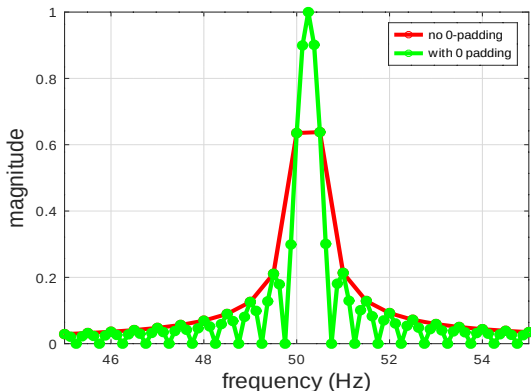
$f_e=512$ Hz, $f_0=50.25$ Hz, $N_{\text{sig}}=1024$, $N_{\text{fft}}=4096$



Effect of the 0-padding (4/4) – see exemples_nplusi.m

Comparizon of the 2 previous FFT

fe=512 Hz, Nsig=1024, f0=50.25 Hz



Windowing – Question 1.

⇒ **Putting into practice matlab!!!** Remember that matlab has an on-line help, and that it provides a lot of functions already done

- ▶ Question 1. Let x be a cosine of frequency 50 *Hz*, of magnitude 1 and of phase $2\pi \frac{50}{512}$. We sample at 512 *Hz*. We analyze a frame of $N = 1024$ samples. Show the obtained magnitude spectrum (zoom between 45 *Hz* and 55 *Hz*).

- ▶ when using a rectangular window or a Hanning window W :

$$W[i] = 0.5 - 0.5 \cos\left(2\pi \frac{i}{N-1}\right) \text{ for } i \in [0 \quad N-1]$$

- ▶ first in linear scale
- ▶ then in *dB* (note: $|X|_{dB} = 20 \log_{10}(|X|)$)
- ▶ the influence of 0-padding will finally be studied

⇒ **interpret the obtained figures!!!**

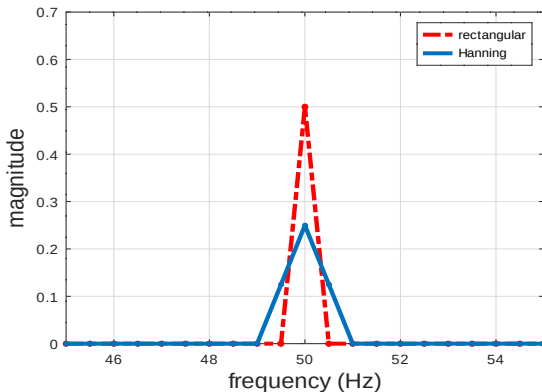
Windowing – solution 1

*ATTENTION: the **figures** to obtain are given on the following slides*

Windowing – solution 1

Linear scale. No 0-padding.

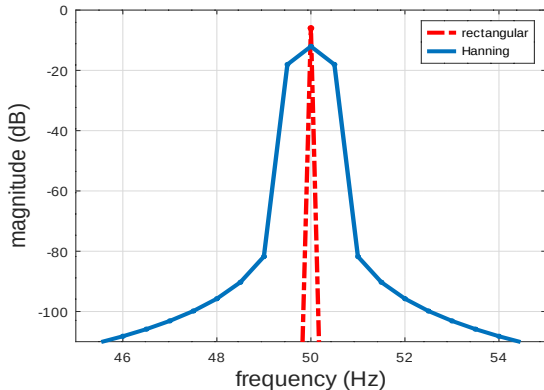
$f_e=512$ Hz, $f_0=50$ Hz, $N_{sig}=1024$, no 0-padding



Windowing – solution 1

In dB. No 0-padding.

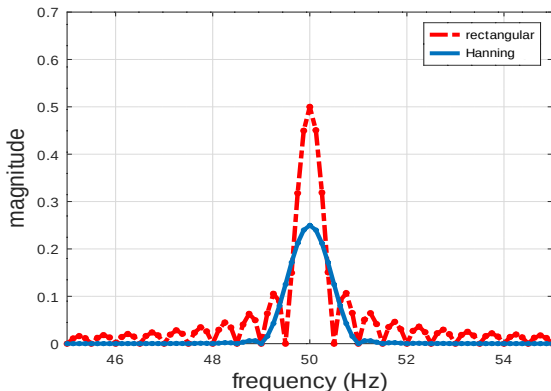
fe=512 Hz, f0=50 Hz, Nsig=1024, no 0-padding



Windowing – solution 1

Linear scale. With 0-padding.

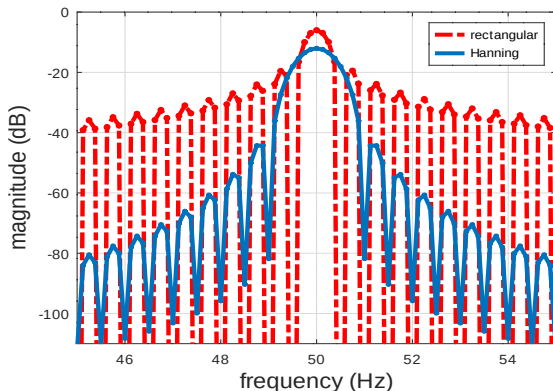
fe=512 Hz, f0=50 Hz, Nsig=1024, NFFT=4096



Windowing – solution 1

In dB. With 0-padding.

fe=512 Hz, f0=50 Hz, Nsig=1024, NFFT=4096



Windowing – solution 1

*ATTENTION: the **matlab** code is given on the following slides*

Windowing – solution 1 (1/2)

```

%% the signal is built
fe = 512;
nsig=1024;
ttt = [1:nsig]/fe; %%% vector of sampling instants (in s)
f0a = 50;
signala = cos(2*pi*f0a*ttt);
%% spectrum computation – no zero-padding
tfft=length(signala);
spectre = abs(fft(signala.*hanning(length(signala)),tfft))/length(signala);
fff = [0:length(spectre)-1]/length(spectre)*fe;
sperec = abs(fft(signala,tfft))/length(signala);
%% figures - no 0-padding
figure(1);clf;grid on;hold on;
plot(fff,sperec,'r-','LineWidth',3);
plot(fff,spectre,'-','LineWidth',3);
xlim([45 55]);
xlabel('frequency (Hz)','FontSize',15);ylabel('magnitude','FontSize',15);
title('fe=512 Hz, f0=50 Hz, Nsig=1024, no 0-padding','FontSize',15);
legend('rectangular','Hanning');
hold off;
figure(2);clf;grid on;hold on;
plot(fff,20*log10(sperec),'r-','LineWidth',3);
plot(fff,20*log10(spectre),'-','LineWidth',3);
xlim([45 55]);ylim([-110 0]); %%% for visualization purposes
xlabel('frequency (Hz)','FontSize',15);ylabel('magnitude (dB)','FontSize',15);
title('fe=512 Hz, f0=50 Hz, Nsig=1024, no 0-padding','FontSize',15);
legend('rectangular','Hanning');hold off;

```

Windowing – solution 1 (2/2)

```
%%% spectrum computation - with 0-padding
tfft=length(signala)*4;
spectre2 = abs(fft(signala.*hanning(length(signala))',tfft))/length(signala);
fff2 = [0:length(spectre2)-1]/length(spectre2)*fe;
sperec2 = abs(fft(signala,tfft))/length(signala);
%%% figures - with 0-padding
figure(3);clf;grid on;hold on;
plot(fff2,sperec2,'r-', 'LineWidth', 3);
plot(fff2,spectre2,'-', 'LineWidth', 3);
xlim([45 55]);
xlabel('frequency (Hz)', 'FontSize',15);
ylabel('magnitude', 'FontSize',15);
title('fe=512 Hz, f0=50 Hz, Nsig=1024, NFFT=4096', 'FontSize',15);
legend('rectangular', 'Hanning');
hold off;
figure(4);clf;grid on;hold on;
plot(fff2,20*log10(sperec2),'r-', 'LineWidth', 3);
plot(fff2,20*log10(spectre2),'-', 'LineWidth', 3);
xlim([45 55]);
ylim([-110 0]);
xlabel('frequency (Hz)', 'FontSize',15);
ylabel('magnitude (dB)', 'FontSize',15);
title('fe=512 Hz, f0=50 Hz, Nsig=1024, NFFT=4096', 'FontSize',15);
legend('rectangular', 'Hanning');
hold off;
```

Spectral analysis/Windowing – Question 1.

- ▶ Question 1. We consider a signal composed of the sum of 3 cosines of frequencies 50 *Hz*, 53.25 *Hz* and 51.75 *Hz*, of magnitudes 1, 0.04 and 0.06, and of phases zero. It is sampled at 512 *Hz*. We analyze a frame of $N = 1024$ samples **centered in $t = 0$** . Show the obtained magnitude spectrum, in *dB*, by calculating the FFT on 8192 points, and this when we use a Rectangular window or a Hanning window (zoom between 45 *Hz* and 59 *Hz*)
- ⇒ **interpret the obtained figures!!!**
- ▶ What is the interest of the windowing?

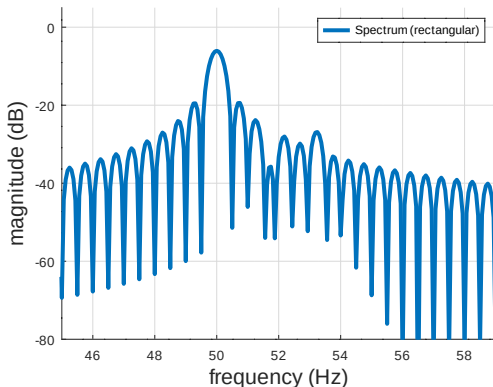
Spectral analysis – solution

*ATTENTION: the **figures** to obtain are given on the following slides*

Spectral analysis – solution

Rectangular windowing (1/6):

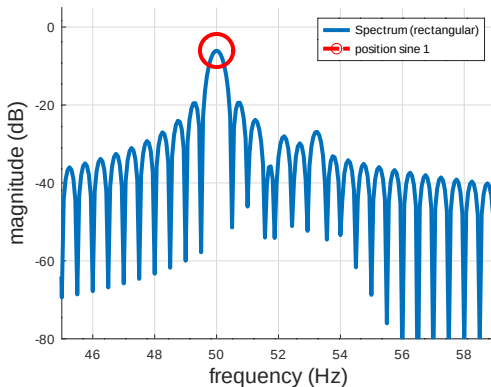
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangular



Spectral analysis – solution

Rectangular windowing (2/6):

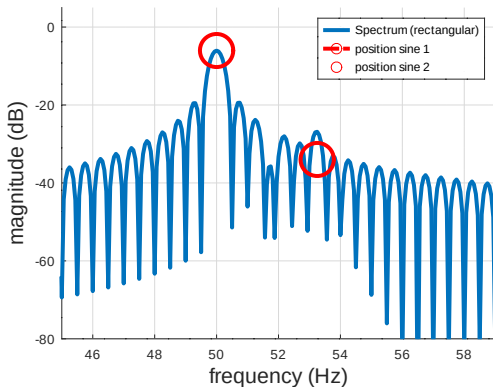
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangula



Spectral analysis – solution

Rectangular windowing (3/6):

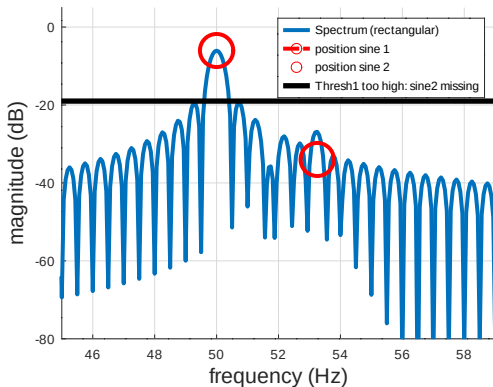
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangula



Spectral analysis – solution

Rectangular windowing (4/6):

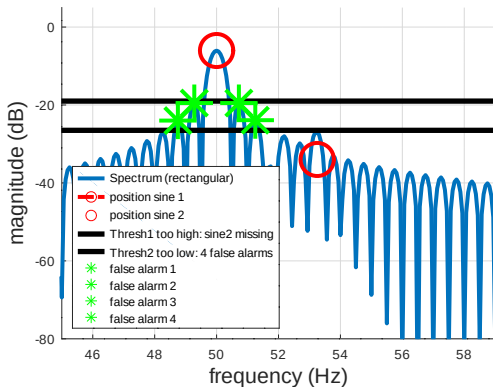
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangula



Spectral analysis – solution

Rectangular windowing (5/6):

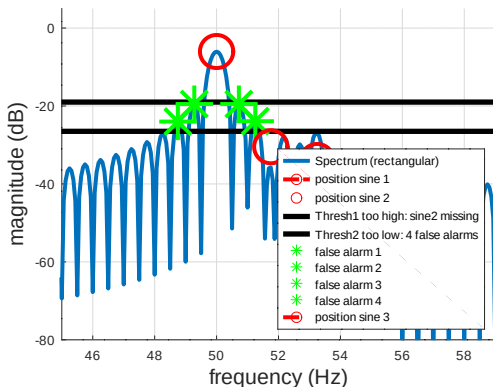
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangula



Spectral analysis – solution

Rectangular windowing (6/6):

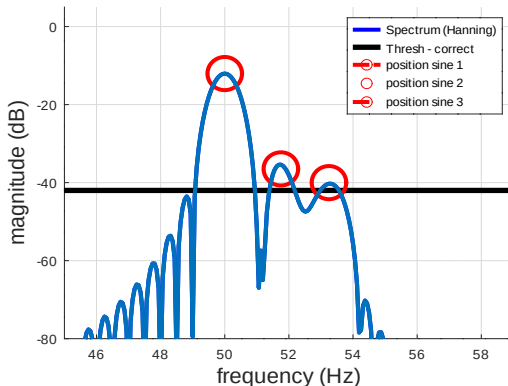
fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Rectangula



Spectral analysis – solution

Hanning windowing:

fe=512 Hz, 3 sinus, Nsig=1024, NFFT=8192, Hanning



Spectral analysis – solution

*ATTENTION: the **matlab** code is given on the following slides*

```
%%% the signal is built
fe = 512;
nsig=1024;
f0a = 50.00;
f0c = 51.75;
f0d = 53.25;
ttt = ([1:nsig]-(1+nsig)/2)/fe;
signala = cos(2*pi*f0a*ttt);
signalc = cos(2*pi*f0c*ttt);
signald = cos(2*pi*f0d*ttt);
aa = 1.00;
ac = 0.06;
ad = 0.04;
signals = aa*signala+ac*signalc+ad*signald;
%%% spectra computation - rectangular window
nfft=4096*2;
win = ones(1,length(signals));
spectre = abs(fft(signals.*win,nfft))/length(signals);
fff = [0:length(spectre)-1]/length(spectre)*fe;
```

```
%% figures
figure(12);
clf;
grid on;
hold on;
xlim([45 59]);
ylim([-80 5]);
nfig=6;
mks=25;
xlabel('frequency (Hz)', 'FontSize', 20);
ylabel('magnitude (dB)', 'FontSize', 20);
title('fe=512 Hz, 3 sines, Nsig=1024, NFFT=8192, Rectangular', 'FontSize', 20);
for ii=1:nfig
    if ii==1
        plot(fff, 20*log10(spectre), '- ', 'LineWidth', 3);
        legend('Spectrum (rectangular)', 'location', 'northeast');
    elseif ii==2
        plot(f0a, 20*log10(aa/2), '-.or', 'markersize', mks, 'LineWidth', 3);
        legend('Spectrum (rectangular)', 'position component 1', 'location', 'northeast');
    elseif ii==3
        plot(f0d, 20*log10(ad/2), 'or', 'markersize', mks, 'LineWidth', 3);
        legend('Spectrum (rectangular)', 'position component 1', 'position component 2', 'location', 'northeast');
```

```
elseif ii==4
    plot([fff(1) 50 fff(length(fff))],[-19 -19 -19], '- -g', 'LineWidth', 3);
    legend('Spectrum (rectangular)','pos. comp.1','pos. comp. 2',...
        'Thresh1 too high : comp.2 missing','location','northeast');
elseif ii==5
    plot([fff(1) 50 fff(length(fff))],[-26.5 -26.5 -26.5], '- -y', 'LineWidth', 3);
    plot(51.25, -23.9, '*y', 'markersize', mks, 'LineWidth', 3);
    plot(48.75, -24.0, '*y', 'markersize', mks, 'LineWidth', 3);
    plot(50.72, -19.5, '*y', 'markersize', mks, 'LineWidth', 3);
    plot(49.28, -19.5, '*y', 'markersize', mks, 'LineWidth', 3);
    legend('Spectrum (rectangular)','pos. comp. 1','pos. comp. 2','Thresh1 too high : comp.2 missing',...
        'Thresh2 too low : 4 false alarms','false alarm 1','false alarm 2','false alarm 3',...
        'false alarm 4','location','northeast');
elseif ii==6
    plot(f0c, 20*log10(ac/2), '- -or', 'markersize', mks, 'LineWidth', 3);
    legend('Spectrum (rectangular)','pos. comp.1','pos. comp. 2','Thresh1 too high : comp.2 missing',...
        'Thresh2 too low : 4 false alarms','false alarm 1','false alarm 2','false alarm 3',...
        'false alarm 4','position component 3','location','northeast');
end;
drawnow;
pause(10);
end;
hold off;
```

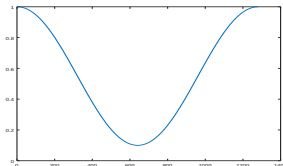
```

%% % spectra computation - Hanning window
win = hanning(length(signals));
spectre = abs(fft(signals.*win,nfft))/length(signals);
fff = [0:length(spectre)-1]/length(spectre)*fe;
%% % figures
figure(13);
clf;grid on;hold on;
plot(fff,20*log10(spectre),'-','LineWidth',3);
plot([fff(1) 50 fff(length(fff))],[-42 -42 -42],'-g','LineWidth',3);
plot(f0a, 20*log10(aa/4), '-.or', 'markersize', mks, 'LineWidth', 3);
plot(f0c, 20*log10(ac/4), 'or', 'markersize', mks, 'LineWidth', 3);
plot(f0d, 20*log10(ad/4), '- -or', 'markersize', mks, 'LineWidth', 3);
plot(fff,20*log10(spectre),'-','LineWidth',3);
xlim([45 59]);
ylim([-80 5]);
xlabel('frequency (Hz)','FontSize',20);
ylabel('magnitude (dB)','FontSize',20);
title('fe=512 Hz, 3 sines, Nsig=1024, NFFT=8192, Hanning','FontSize',20);
legend('Spectrum (Hanning)','Thresh - correct','pos. component 1','pos. component 2','pos. component 3');
hold off;

```

Inverse windowing

- ▶ We can try to do the opposite of what classic windows do: instead of decreasing the frequency resolution to increase the resolution in magnitude, we want to increase the resolution in frequency at the cost of the magnitude resolution
- ▶ radar specialists sometimes use this type of window:



⇒ see fenetreinverse.m

Daniell estimator

1946. We average the periodogram **in the frequency domain** over a small interval around each analysis frequency:

$$\blacktriangleright \hat{S}_{\text{dan}}(\nu) = \frac{1}{2P+1} \sum_{\nu_i=\nu-P/fe}^{\nu+P/fe} \hat{S}(\nu_i)$$

$\Rightarrow$ the variance is divided by a factor smaller than $2P+1$ (the frequency decorrelation is not perfect)

my code: mydaniell.m

Bartlett estimator

1948. The average is no longer computed in the frequency domain:

- ▶ the frame of N samples is split into K non-overlapping segments of M samples, so that $N = KM$
- ▶ the periodogram is computed on each of these K segments
- ▶ the average of the K periodograms is computed

⇒ the variance is divided by a factor close to K

my code: `mybartlett.m`

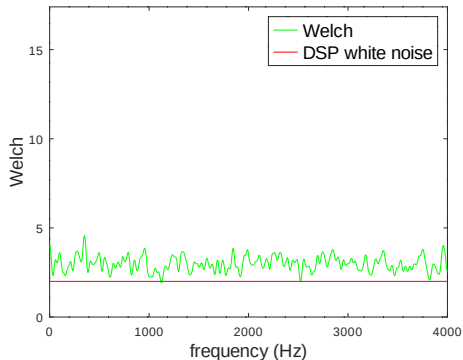
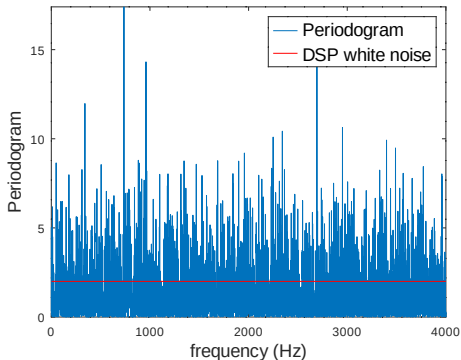
Welch estimator (1/2)

1967. Bartlett generalization: this time, the K' segments can overlap and a ponderation window is possibly used on each segment (K' is greater than Bartlett's K)

⇒ the variance is divided by a factor close to K'

my code: mywelch.m

Welch estimator (2/2)



⇒ see mywelchbruit.m

Correlogram (Blackman-Tukey)

- ▶ The periodogram and the PSD are equal, at the limit, also:

$$\hat{X}(\nu) = \frac{1}{N} | \text{FT}(x) |^2$$

- ▶ And the PSD is also the FT of the autocorrelation (Wiener-Kintchine theorem, 1930):

$$\hat{X}(\nu) = | \text{FT}(\gamma_x(n)) |$$

- ▶ the idea of the correlogram, is to use only the central autocorrelation coefficients to calculate the PSD, since the aurocorrelation coefficients are more and more badly estimated as the time-lag increases
- ▶ in addition, we can use either the biased estimator or the unbiased estimator of the autocorrelation

⇒ see mycorre.m

Plan

- ▶ Part 1: Introduction
- ▶ Part 2: More about standard (non-parametric) spectral analysis (periodogram...)
- ▶ **Part 3: Parametric spectral analysis: AR/MA/ARMA**
- ▶ Part 4: Advanced spectral analysis: MUSIC, Pisarenko, Prony
- ▶ Part 5: Laplace, Z, Bode...
- ▶ Part 6: Filtering: analog versus digital
- ▶ Part 7: Advanced spectral analysis: Wavelets
- ▶ Part 8: Matching/basis pursuit
- ▶ Part 9: Sparsity
- ▶ Part 10: Random signals: temporal/spectral properties

Part 3 – Parametric spectral analysis

Stéphane Rossignol

September 14, 2026

Sub-Plan

- ▶ Introduction to parametric models

Introduction

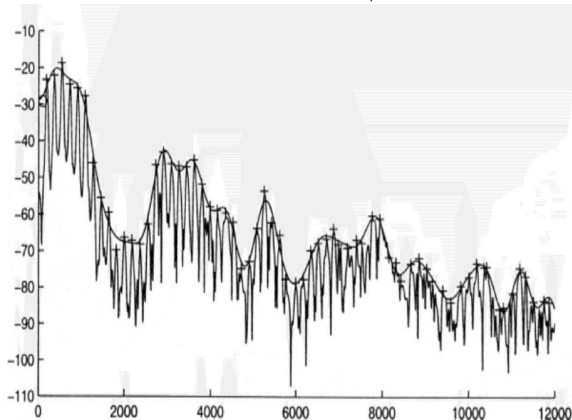
- ▶ in numerous applications, we try to retrieve information from N temporal (or spatial) observations $\{x(1) \dots x(N)\}$
- ▶ although the time (or space) and frequency domains are dual, information is often more intelligible in the frequency domain
- ▶ hence the need to develop tools to perform the spectral analysis of a signal
- ▶ spectral analysis is often used as pre-processing for classification, recognition, compression, filtering, detection, etc., of signals; it's important to understand well what's going on!!!!

Introduction

- ▶ we assume here (parametric methods) that the signal belongs to a family of signals which depends on a set of parameters Θ
 - ▶ the first step is to estimate these parameters Θ
 - ▶ then, in a second step, an estimate of the spectrum can be obtained (by duality)
- ▶ these methods are also said to be high-resolution, because they are able to exceed the frequency resolution of non-parametric methods (particularly for N small)
- ▶ they also make it possible to estimate the spectral envelopes

Introduction

We want to catch the spectral lines and/or the spectral envelope:



Introduction

- ▶ the fundamental problem of spectral analysis, is the principle of uncertainty of Gabor (it is the equivalent for signal processing of the Heisenberg's principle of uncertainty)
- ▶ the analyzed signals are stationary/pseudo-stationary only over very short time intervals (of the order of $\Delta T = 40 \text{ ms}$ for sounds, for instance)
- ▶ therefore, we can only efficiently analyze signal frames of this size order ΔT
- ▶ thus, we greatly limit the frequency resolution Δf that we could hope for

Introduction

- ▶ it means, if we admit that the signal is composed of spectral lines (frequencies f_i , amplitudes a_i , phases ϕ_i), that we cannot hope to completely separate lines that are too close in f
- ▶ indeed, if we call Δf the frequency distance between two lines, Gabor says that $\Delta T \Delta f = \text{constant}$
- ▶ we can exceed this limit, for example with the high-resolution methods that we will see below, but there is a **cost**
- ▶ this **cost** is concentrated on the estimation of the amplitudes a_i and phases ϕ_i , in particular
- ▶ as well as on the complexity of the estimation procedures (values of various orders, controlling the number of parameters)

Bibliography

- ▶ Steven M. Kay and Stanley Lawrence Marple Jr. “Spectrum analysis – a modern perspective”, Proceedings of the IEEE, Volume 69, pages 1380-1419, novembre 1981
- ▶ Gilles Fleury. Analyse spectrale – Méthodes non-paramétriques et paramétriques, Ellipses, 2001
- ▶ J. E. Rooijackers, “Algorithms for Speech Coding Systems Based on Linear Prediction”, EUT Report 92-E-260, 1992
 - ▶ a practitioner's point of view
 - ▶ available on the web

Sub-Plan

- ▶ Introduction to parametric models
- ▶ AR models

ARMA models, in general

Some deterministic or random processes, encountered in the real world, can be modeled in this way:

$$x_n = \sum_{l=0}^q b_l n_{n-l} - \sum_{k=1}^p a_k x_{n-k}$$

where x_n is the signal to be modeled, a_k and b_l the parameters to be determined, and n_n a Gaussian white noise (generating noise)

AR models – AR=Autoregressive

- ▶ in this case (AR): $q = 0$ and $b_0 = 1$
- ▶ the model is then: $x_n = - \sum_{k=1}^p a_k x_{n-k} + n_n$ M1
- ▶ we must estimate the parameters a_k , minimizing the variance σ^2 of the noise n
- ▶ several approaches are possible (a large number of approaches are possible, in fact!!!)

Algorithms

- ▶ so, now we have to solve the problem, this directly from the signal x ; or from the autocorrelation coefficients $\hat{R}_{xx}$ **estimated**, therefore imperfect
 - ▶ they are estimated and to a certain extent poorly estimated because the observation horizon is limited to N samples
- ▶ the three algorithms that we are going to examine in (some) extent are:
 - ▶ the one of **Levinson-Durbin** (some just call it of Levinson; others – even – of Yule-Walker)
 - ▶ the one of **Burg**
 - ▶ the one of **Marple**

Algorithm 1 – Levinson-Durbin

- ▶ we first estimate the autocorrelation coefficients $\hat{R}_{xx}$
- ▶ from these autocorrelation coefficients, it is possible to obtain the AR coefficients
- ▶ the preliminary step consists in obtaining the Yule-Walker equations, which relate the AR parameters to the autocorrelation coefficients of x , R_{xx}

Yule-Walker equations

- ▶ we have:

$$\begin{aligned}
 R_{xx}(k) &= E[x_{n+k}x_n^*] \\
 &= E\left[x_n^* \left(-\sum_{l=1}^p a_l x_{n-l+k} + n_{n+k}\right)\right] \\
 &= -\sum_{l=1}^p a_l R_{xx}(-l+k) + E[n_{n+k}x_n^*]
 \end{aligned}$$

- ▶ and the relation **M1** between n_n and x_n can also be considered as a filtering processing, with n_n the input (generating noise) and x_n the output

Yule-Walker equations

- ▶ the transfer function of this stable, linear and causal filter is:

$$H(z) = \frac{1}{A(z)} = \frac{1}{\sum_{m=0}^p a_m z^{-m}} \text{ with } a_0 = 1$$

- ▶ we call h_l the impulse response of this filter
- ▶ since the system is causal, $h_l = 0 \forall l < 0$
- ▶ we get then:

$$E[n_{n+k} x_n^*] = E \left[n_{n+k} \sum_{l=0}^{+\infty} h_l^* n_{n-l}^* \right]$$

Yule-Walker equations

- ▶ this becomes:

$$E[n_{n+k}x_n^*] = \sum_{l=0}^{+\infty} h_l^* E[n_{n+k}n_{n-l}^*]$$

- ▶ yet n_n is a white noise; its variance is σ^2 (it will also have to be estimated) and of course its autocorrelation is:

$$E[n_{n+k}n_{n-l}^*] = \sigma^2 \delta_{k+l}$$

- ▶ then:

$$\begin{aligned} E[n_{n+k}x_n^*] &= \sigma^2 h_{-k}^* \\ &= \begin{cases} 0 & \text{for } k > 0 \text{ and} \\ \sigma^2 h_0^* & \text{for } k = 0 \end{cases} \end{aligned}$$

Yule-Walker equations

- ▶ yet: $h_0 = \lim_{z \rightarrow +\infty} H(z) = 1$ (initial value theorem)
- ▶ thus, we get (it is one of the forms of Yule-Walker equations):

$$R_{xx}(k) = \begin{cases} -\sum_{l=1}^p a_l R_{xx}(k-l) & \text{for } k > 0 \\ -\sum_{l=1}^p a_l R_{xx}(k-l) + \sigma^2 & \text{for } k = 0 \end{cases}$$

Yule-Walker equations

- ▶ to determine the AR parameters, just choose p equations from the set above
- ▶ the set of equations which requires the smallest offsets k is the one obtained by taking $k = 1, 2 \dots p$
 - ▶ it matches those who are best estimated!
- ▶ to estimate σ^2 , an additional equation is needed
- ▶ one must also remember that $R_{xx}(-m) = R_{xx}^*(m)$

Yule-Walker equations

- thus, in matrix form, it is written:

$$\begin{bmatrix} R_{xx}(0) & R_{xx}(-1) & \dots & R_{xx}(-(p-1)) \\ R_{xx}(1) & R_{xx}(0) & \dots & R_{xx}(-(p-2)) \\ \vdots & \vdots & \ddots & \vdots \\ R_{xx}(p-1) & R_{xx}(p-2) & \dots & R_{xx}(0) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = - \begin{bmatrix} R_{xx}(1) \\ R_{xx}(2) \\ \vdots \\ R_{xx}(p) \end{bmatrix}$$

- or if we incorporate the estimate of σ^2 :

$$\begin{bmatrix} R_{xx}(0) & R_{xx}(-1) & \dots & R_{xx}(-p) \\ R_{xx}(1) & R_{xx}(0) & \dots & R_{xx}(-(p-1)) \\ \vdots & \vdots & \ddots & \vdots \\ R_{xx}(p) & R_{xx}(p-1) & \dots & R_{xx}(0) \end{bmatrix} \begin{bmatrix} 1 \\ a_1 \\ \vdots \\ a_p \end{bmatrix} = \begin{bmatrix} \sigma^2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Algorithm 1 – Levinson-Durbin

- ▶ the Levinson-Durbin algorithm is an algorithm of inversion of the square matrix above
 - ▶ it requires $o(p^2)$ operations $\Rightarrow o(p^3)$ for the Gaussian pivot
- ▶ it is not only effective: it also reveals interesting properties of AR processes
- ▶ it is iterative: we start with an order p equal to 1 and we go to the desired p order
 - ▶ the order p at which to stop is also not known
 - ▶ we will see later techniques for deciding in which order p to stop
 - ▶ we get these sets of parameters: $\{a_{1,1}, \sigma_1^2\}$, $\{a_{2,1}, a_{2,2}, \sigma_2^2\}$, ..., $\{a_{p,1}, a_{p,2}, \dots, a_{p,p}, \sigma_p^2\}$
 - ▶ note the additional index, which indicates the order p reached

Algorithm 1 – Levinson-Durbin

- ▶ at the iteration 1 ($p = 1$), we get:

$$a_{1,1} = -\frac{\hat{R}_{xx}(1)}{\hat{R}_{xx}(0)} \quad \text{E1}$$

$$\sigma_1^2 = (1 - |a_{1,1}|^2) \hat{R}_{xx}(0) \quad \text{E2}$$

- ▶ which is easily obtained from:

$$\begin{bmatrix} \hat{R}_{xx}(0) & \hat{R}_{xx}^*(1) \\ \hat{R}_{xx}(1) & \hat{R}_{xx}(0) \end{bmatrix} \begin{bmatrix} 1 \\ a_{1,1} \end{bmatrix} = \begin{bmatrix} \sigma^2 \\ 0 \end{bmatrix}$$

- ▶ from line 2 we get the estimate of $a_{1,1}$, and also $\hat{R}_{xx}(1) = -a_{1,1}\hat{R}_{xx}(0)$ and then $\hat{R}_{xx}^*(1) = -a_{1,1}^*\hat{R}_{xx}(0)$ (because $\hat{R}_{xx}(0)$ is real)
- ▶ from line 1 we get $\sigma^2 = \hat{R}_{xx}(0) + a_{1,1}\hat{R}_{xx}^*(1)$, etc.

Algorithm 1 – Levinson-Durbin

- ▶ at iterations $k = 2, 3 \dots$ we get:

$$a_{k,k} = - \frac{\hat{R}_{xx}(k) + \sum_{l=1}^{k-1} a_{k-1,l} \hat{R}_{xx}(k-l)}{\sigma_{k-1}^2} \quad \text{E3}$$

$$a_{k,i} = a_{k-1,i} + a_{k,k} a_{k-1,k-i}^* \text{ for } i < k \quad \text{E4}$$

$$\sigma_k^2 = (1 - |a_{k,k}|^2) \sigma_{k-1}^2 \quad \text{E5}$$

- ▶ note: this is the Levinson-Durbin recursion

Companion calculations – 1. Autocorrelation estimation

- ▶ unbiased estimator (up to $m \leq N - 1$):

$$\hat{R}_{xx}(m) = \frac{1}{N - m} \sum_{n=0}^{N-m-1} x_{n+m} x_n^*$$

- ▶ biased estimator (up to $m \leq N - 1$):

$$\hat{R}'_{xx}(m) = \frac{1}{N} \sum_{n=0}^{N-m-1} x_{n+m} x_n^* \quad \text{E6}$$

- ▶ the second form is most often used, as for a lot of data it tends to have a smaller quadratic error

Companion calculations – 2. PSD estimation

- ▶ n is a white noise, thus its PSD is equal to σ^2/f_e
- ▶ the interference formula says that the PSD of x compared to that of n is:

$$P_{xx}(z) = H(z)H^*(1/z^*)P_n(z) = \frac{1}{A(z)A^*(1/z^*)}P_n(z)$$

- ▶ it is estimated for $z = \exp(j2\pi f/f_e)$ for $-f_e/2 \leq f \leq f_e/2$
- ▶ thus, we get:

$$P_{xx}(f) = |H(f)|^2 P_n(f) = \frac{\sigma^2}{f_e \left| 1 + \sum_{k=1}^p a_k \exp(-j2\pi k f / f_e) \right|^2} \quad \text{E7}$$

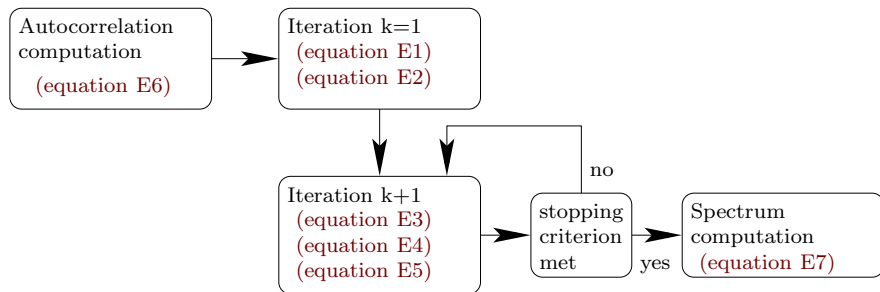
Algorithm 1 – Levinson-Durbin – Notes

- ▶ the algorithm provides the AR parameters for all models of orders lower than p and which best match the data
- ▶ we could use this as a stop criterion:
 - ▶ σ_k^2 decreasing with k (because $|a_{k,k}| \leq 1$), when σ_k^2 is small enough, we stop the algorithm
 - ▶ moreover, if the process is really AR of order p , then we have $a_{k,k} = 0$ and $\sigma_k^2 = \sigma^2$ for $k > p$
 - ▶ unfortunately, it does not turn out like this in reality, as we will see

Algorithm 1 – Levinson-Durbin – Notes

- ▶ the parameters $\{a_{1,1}, a_{2,2}, \dots, a_{p,p}\}$ are called the reflection coefficients
 - ▶ we also note them $\{k_1, \dots, k_p\}$
 - ▶ if all the $|k_i|$ are equal to ≤ 1 , the used autocorrelation coefficients $\hat{R}_{xx}$ are correct
 - ▶ then, all the poles of $\frac{1}{A(z)}$ are inside the unit circle (or on it)
 - ▶ if we have $|k_k| = 1$ for a given k , then the signal is purely harmonic (composed of sines)

Algorithm 1 – Levinson-Durbin – Summary



```
function [aa, sigma2, ref, ff, mydsp] = mylevinsondurbin (xx, pp, fe)

acf = xcorr(xx, pp+1, 'biased');
acf(1:pp+1) = []; %% we remove the negative part
acf(1) = real(acf(1)); %% Levinson-Durbin requires c(1)==conj(c(1))

ref = zeros(pp,1);
gg = -acf(2)/acf(1);
aa = [ gg ];
sigma2 = real( ( 1 - gg*conj(gg)) * acf(1) ); %% real: removes any potential residual imaginary part
ref(1) = gg;
for tt = 2 : pp
    gg = -(acf(tt+1) + aa * acf(tt-1:2))' / sigma2;
    aa = [ aa + gg*conj(aa(tt-1:1:1)), gg ];
    sigma2 = sigma2 * ( 1 - real(gg*conj(gg)) ) ;
    ref(tt) = gg;
end;
aa = [1, aa];

interm2=-j*2*pi/fe*[1:pp];
ff=-fe/2:10:fe/2;
for ii=1:length(ff)
    interm=1.+aa(2:end)*exp(interm2*ff(ii))';
    mydsp(ii) = sigma2./(interm.*conj(interm)); %%% power spectral density
end;

figure(1);
clf;
grid on;
hold on;
plot(ff,mydsp);
hold off;
```

Algorithm 1 – Levinson-Durbin – First tests

We are going to do some tests, with simple signals (sounds) to verify that we are indeed obtaining interesting power spectral densities (PSD). The 4 tests considered here are:

- ▶ we have a pure sine $\Rightarrow$ can we manage to highlight it?
- ▶ we have 2 sines, close in frequency $\Rightarrow$ can we manage to discriminate them?
- ▶ we have a composed signal: a sum of harmonic sines $\Rightarrow$ can we manage to get a spectral envelop, modeling this signal, and/or the n partials?
- ▶ we have a more complicated signal: a pink noise $\Rightarrow$ can we manage to get a spectral envelop, modeling this signal?

First tests : parameters

- ▶ Parameters of the 1st test: $f_e=32000$ Hz; frequency of the sine 440 Hz; angle, random between 0 et 2π ; frame size 1280 samples (40 ms)
- ▶ Parameters of the 2nd test: $f_e=32000$ Hz; 2 sines of frequency 440 Hz and 466 Hz; same magnitude; frame size 1280 samples (40 ms)
- ▶ Parameters of the 3d and 4th tests: $f_e=32000$ Hz; $f_0 = 220$ Hz; 20 partials; magnitudes: following a gaussian centered on $f_{max} = 2500$ Hz and of standard deviation 1500; angle, random between 0 et 2π ; frame size 1280 samples (40 ms)
- ▶ Parameters of the last test: $f_e=32000$ Hz; central frequency of the pink noise, $f_c = 2500$ Hz; band width: 1500 Hz; frame size 1280 samples (40 ms)

Algorithms – Summary of the results

	Levinson-Durbin	FFT
1 sine	$p \geq 4$	yes
2 sines	$p = 250$: 63 % $p = 265$: 88 % $p = 300$: 100 %	Blackman 32 % Hanning 40 % (Rectangular 64 % other meth.)
Envelope	norm 1 : $e > 0.318$ norm 2 : $e > 0.305$	detection of peaks to do
20 partials	yes ($p = 200$)	yes
pink noise	norm 1 : $e > 0.568$ norm 2 : $e > 0.367$ norm 3 : $e > 0.378$	$e \simeq 0.5^1$

¹see scriptpinkfft.m

Algorithm 2 – Burg

- ▶ this time, we no longer use the estimated autocorrelation coefficients, but the fact that the AR model and the linear prediction are linked
- ▶ indeed, we can prove that $\hat{x}_n = -\sum_{k=1}^p a_k x_{n-k}$ is the best estimate of x_n , within the framework of the linear prediction (see references)
- ▶ the prediction error for the sample n is then ($p+1 \leq n \leq N$):

$$e_p(n) = x_n - \hat{x}_n = \sum_{k=0}^p a_k x_{n-k} \quad (\text{with } a_0 = 1)$$

Algorithm 2 – Burg

- ▶ $e_p(n)$ is the **forward** prediction error: we predict x_n from the past samples $\{x_{n-p} \dots x_{n-1}\}$
- ▶ we can define in the same way the **backward** prediction error $r_p(n)$: we predict (“postdict”) x_{n-p} from the future samples $\{x_{n-p+1} \dots x_n\}$
- ▶ we get then ($p+1 \leq n \leq N$):

$$r_p(n) = x_{n-p} - \hat{x}_{n-p} = \sum_{k=0}^p a_k x_{n-p+k} \quad (\text{with } a_0 = 1)$$

Algorithm 2 – Burg

- ▶ the whole idea of Burg's algorithm consists in minimizing the sum of the two squared prediction errors
- ▶ note that they are both defined between $n = p + 1$ and $n = N$
- ▶ moreover, Burg's algorithm is iterative, like Levinson-Durbin's: we increase the order gradually
- ▶ then, we again get the sets of parameters: $\{a_{1,1}, \sigma_1^2\}$, $\{a_{2,1}, a_{2,2}, \sigma_2^2\}$, ..., $\{a_{p,1}, a_{p,2}, \dots, a_{p,p}, \sigma_p^2\}$

Algorithm 2 – Burg

- ▶ then, we want to minimize this sum:

$$E_p = \sum_{n=p+1}^N \{e_p^2(n) + r_p^2(n)\}$$

- ▶ note: if we just minimize $\sum_{n=p+1}^N e_p^2$, we fall back on the Levinson-Durbin's algorithm; and besides, it is by passing through this that we also prove the assertion “the best estimate of x_n in the linear prediction frame is $-\sum_{k=1}^p a_k x_{n-k}$ ”

Algorithm 2 – Burg

- ▶ let us now take again the recursion of Levinson-Durbin, which says (k being the current order: it increases recursively):

$$a_{k,k} = -\frac{\hat{R}_{xx}(k) + \sum_{l=1}^{k-1} a_{k-1,l} \hat{R}_{xx}(k-l)}{\sigma_{k-1}^2}$$

$$a_{k,i} = a_{k-1,i} + a_{k,k} a_{k-1,k-i}^* \text{ for } i < k$$

- ▶ by posing $\gamma_{k+1} = \frac{\sum_{l=0}^k a_{k,l} \hat{R}_{xx}(k+1-l)}{\sigma_k^2}$ (γ takes an important place), it is easier to write this this way:

$$a_{k,k} = -\gamma_k \quad \text{F1}$$

$$a_{k,i} = a_{k-1,i} - \gamma_k a_{k-1,k-i}^* \text{ for } i < k \quad \text{F2}$$

Algorithm 2 – Burg

- ▶ by manipulating somewhat the equations of forward and backward prediction errors, as well as the Levinson-Durbin recursivity equations, we obtain:

$$e_k(n) = e_{k-1}(n) - \gamma_k r_{k-1}(n-1) \quad \text{F3}$$

$$r_k(n) = r_{k-1}(n-1) - \gamma_k e_{k-1}(n) \quad \text{F4}$$

- ▶ by substituting these equations in the equation we obtained for E_p , we can write:

$$E_k = D_k \gamma_k^2 - 2N_k \gamma_k + D_k$$

that we are going to want to minimize with respect to γ_k (these γ correspond to the inverses of the reflection coefficients, of course)

Algorithm 2 – Burg

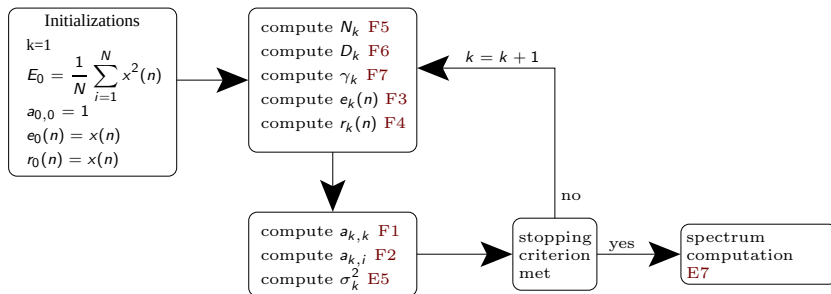
- ▶ and we have for N_k and D_k :

$$N_k = 2 \sum_{n=k+1}^N e_{k-1}(n) r_{k-1}(n-1) \quad \text{F5}$$

$$\text{and } D_k = \sum_{n=k+1}^N \{e_{k-1}^2(n) + r_{k-1}^2(n-1)\} \quad \text{F6}$$

- ▶ it remains to find the derivative of E_k with respect to γ_k , and to cancel this derivative
- ▶ we easily get: $\gamma_k = \frac{N_k}{D_k} \quad \text{F7}$
- ▶ and now we have everything we need to find the AR parameters

Algorithm 2 – Burg



```
function [aa, sigma2, kk, ff, mydsp] = myburg(xx, pp, fe)
% Initializations
N = length(xx);
xx = xx(:);
ef = xx;
eb = xx;
aa = 1;
EE = xx'*xx./N;
kk = zeros(1, pp); % Pre-allocation of 'kk' (to improve execution speed)

for tt=1:pp % Compute the following reflection coefficient
    efp = ef(2:end);
    ebp = eb(1:end-1);
    num = -2.*ebp*efp; %%% it's -Nk (F5)
    den = efp*efp+ebp*ebp; %%% it's Dk (F6)
    kk(1,tt) = num./den; %%% it's  $-\gamma$  in the slides (F7)
    % Updating 'forward' and 'backward' prediction errors
    ef = efp + kk(tt)*ebp; %%% (F3)
    eb = ebp + kk(tt)*efp; %%% (F4)
    aa=[aa;0] + kk(1,tt)*[0;conj(flipud(aa))]; % Updating the AR coefficients (F1 and F2)
    EE(tt+1) = (1 - kk(1,tt)*kk(1,tt))*EE(tt); % Global prediction error update (E5)
end
aa = aa(:).';
sigma2=EE(end); %%% not necessarily well estimated; note: it's EE

%% power spectral density
interm2=-j*2*pi/fe*[1:pp];
ff=-fe/2:10:fe/2;
for ii=1:length(ff)
    interm=1.+aa(2:end)*exp(interm2*ff(ii))';
    mydsp(ii) = sigma2./(interm.*conj(interm));
end;
```

Algorithms – Summary of the results

	Levinson-Durbin	Burg	FFT
1 sine	$p \geq 4$	$p \geq 2$	yes
2 sines	$p = 250$: 63 % $p = 265$: 88 % $p = 300$: 100 %	$p = 4$: 99.68 % $p = 5$: 100 % $p = 6$: 97 % $p = 7$: 7 %	Blackman 32 % Hanning 40 % (Rectangular 64 % other meth.)
Envelope	$norm1 : e > 0.318$ $norm2 : e > 0.305$	not quite effective	detection of peaks to do
20 partials	yes ($p = 200$)	yes ($p = 100$)	yes
pink noise	$norm1 : e > 0.568$ $norm2 : e > 0.367$ $norm3 : e > 0.378$	$norm1 : e > 0.708$ $norm2 : e > 0.456$ $norm3 : e > 0.493$	$e \simeq 0.5$

Algorithm 3 – Marple

- ▶ this time, we're not going to look at the algorithm in detail: we could spend at least one session on it
- ▶ therefore, for those interested, see the 3rd reference
- ▶ we will only examine the important points, i.e. what makes the difference compared to the two previous algorithms
- ▶ it's again an iterative algorithm: we start from the order 1, and we go up to the desired order p , obtaining all the parameters AR at the intermediate orders k

Algorithm 3 – Marple

- ▶ **point 1.:** for Burg, we saw that we canceled the derivative of E_k with respect to γ_k
- ▶ γ_k is the reflection coefficient obtained at the order $k \Rightarrow$ it's equal to $-a_{k,k}$
- ▶ here, we cancel the derivative of E_k with respect to all the parameters $a_{k,i}$ obtained at order k
- ▶ we obtain a system of the type: $T_k \mathbf{a}_k = \mathbf{e}_k$, that we must solve
- ▶ note 1: of course, the vectors $\mathbf{a}_k$ and $\mathbf{e}_k$ always follow the same notations (see Burg algorithm, for $\mathbf{e}_k$)
- ▶ note 2: the matrix T_k is obtained from the samples of the signal x and only from them

Algorithm 3 – Marple

- ▶ **point 2.:** two prediction errors must be added and they need to be minimized too (otherwise, we fall back on an another existing method):

$$E'_k = \sum_{n=k+1}^{N-1} \{e_k^2(n+1) + r_k^2(n)\}$$

$$E''_k = \sum_{n=k+1}^{N-1} \{e_k^2(n) + r_k^2(n+1)\}$$

- ▶ we obtain systems of the type: $T'_k \mathbf{a}'_k = \mathbf{e}'_k$ and $T''_k \mathbf{a}''_k = \mathbf{e}''_k$, that must be solved jointly with the previous system

Algorithm 3 – Marple

- ▶ all the rest consists in developing and manipulating these equations to obtain a recursion of the type of that of Levinson-Durbin, which is exploitable (it is very twisted and very long to do)
- ▶ **point 3.:** the obtained recursion is:

$$a_{k,k} = -\gamma_k \quad \text{G1}$$

$$a_{k,i} = a'_{k-1,i} - \gamma_k a'^*_{k-1,k-i} \text{ for } i < k \quad \text{G2}$$

- ▶ what should be noted is the replacement of the terms $a_{k-1,i}$ by the “shifted in time” terms $a'_{k-1,i}$

Sub-Plan

- ▶ Introduction to parametric models
- ▶ AR models
- ▶ Performances, choice of the order, MA and ARMA models

Algorithms – Summary of the results

	Levinson-Durbin	Burg	Marple	FFT
1 sine	$p \geq 4$	$p \geq 2$	$3 \leq p \leq 10$ $\sigma^2 = 1e^{-4}$ $4 \leq p \leq 40$ $\sigma^2 = 1e^{-3}$	yes
2 sine	$p = 250$: 63 % $p = 265$: 88 % $p = 300$: 100 %	$p = 4$: 99.68 % $p = 5$: 100 % $p = 6$: 97 % $p = 7$: 7 %	$p = 90$: 16 % $p = 100$: 83 % $p = 110$: 93 % $p = 120$: 86 %	Blackman 32 % Hanning 40 % (Rect. 64 % other meth.)
Envelope	$norm1 : e > 0.318$ $norm2 : e > 0.305$	pas très efficace	$norm1 : e > 0.337$ $norm2 : e > 0.310$	detection of peaks to do
20 partials	yes ($p = 200$)	yes ($p = 100$)	yes ($p = 140$)	yes
pink noise	$norm1 : e > 0.568$ $norm2 : e > 0.367$ $norm3 : e > 0.378$	$norm1 : e > 0.708$ $norm2 : e > 0.456$ $norm3 : e > 0.493$	$norm1 : e > 0.520$ $norm2 : e > 0.344$	$e \simeq 0.5$

Essential differences between these various algorithms

- ▶ **Levinson-Durbin**: iterative inversion of the autocorrelation matrix
- ▶ **Burg**: minimization of the prediction error (forward + backward) compared to the reflection coefficient obtained at each iteration:

$$\frac{\partial E_p}{\partial a_{k,k}} = 0$$

- ▶ **Marple**: minimization of the prediction error (forward + backward) with respect to all the AR parameters obtained at each iteration:

$$\frac{\partial E_p}{\partial a_{k,i}} = 0 \quad \forall i$$

- ▶ **Morf**: minimization of the prediction error at each iteration, but this time considering that the AR coefficients are not the same for the backward prediction and for the forward prediction
- ▶ and the list could go on and go on...

Some salient points (negative) of the 3 tested methods

▶ Levinson-Durbin

- ▶ estimation of the autocorrelation coefficients (imperfect and possibly time consuming)
- ▶ resolution not necessarily extraordinary (to be tested further)

▶ Burg

- ▶ possible doubling of the lines, in particular when the signal to noise ratio (SNR) is bad (to be tested further)
- ▶ estimation of the positions of the peaks/lines depending on the phase, i.e. bias (to be tested further)

▶ Marple: uncertain stability (this, you can easily verify)

▶ All of them:

- ▶ how to choose the order?
- ▶ decreasing resolution with SNR (to be tested)
- ▶ amplitudes of the lines not related to those of the actual sines

Some salient points (positives) of these methods

- ▶ In terms of resolution, Marple showed that 2 sines, with an additive noise, could be separated for Δf of this order:

$$\Delta f \simeq \frac{1.03}{p[(p+1)\text{SNR}]^{0.31}} \text{ for the AR models}$$

$$\Delta f \simeq \frac{0.86}{N} \text{ for the periodogram}$$

- ▶ algorithms simple and fast (!)
- ▶ results no worse than those of non-parametric methods
- ▶ exploitable results!

Order p selection

- ▶ in reality, as already mentioned extensively, we do not know which order p to choose
- ▶ there is a bunch criteria supposed to help decide what order to take (none working completely satisfactorily)
- ▶ it should be noted that during the iterative computation of the AR parameters, we obtain at each iteration k the variance of the generating white noise $\hat{\sigma}_k^2$: we will need these successive variances in the following criteria
- ▶ note: all the following criteria are to be minimized according to k
- ▶ I will let you test them for yourself

Order p selection

- **[1.]** the first criterion is FPE (Final Prediction Error); we can compute this term, where N is the number of samples of the signal, at each iteration k :

$$FPE(k) = \frac{N + (k + 1)}{N - (k + 1)} \hat{\sigma}_k^2$$

- **[2.]** then there is AIC (Akaike Information Criterion); we can compute this term, at each iteration k :

$$AIC(k) = \frac{2k}{N} + \log(\hat{\sigma}_k^2)$$

Order p selection

- **[3.]** then there is MDL (Minimum Description Length); we can compute this term, at each iteration k :

$$MDL(k) = \frac{k \log N}{N} + \log (\hat{\sigma}_k^2)$$

- **[4.]** then there is CAT (Criterion Autoregressive Transfer); we can compute this term, at each iteration k :

$$CAT(k) = \frac{1}{N} \sum_{i=1}^k \left(\frac{N-i}{N} \frac{1}{\hat{\sigma}_i^2} \right) - \frac{N-k}{N} \frac{1}{\hat{\sigma}_k^2}$$

MA models

- ▶ the model is:

$$x_n = \sum_{l=0}^q b_l n_{n-l}$$

- ▶ we must estimate the parameters b_l
- ▶ this expression directly gives the impulse response of the filter associated with the MA modelization: $h_i = b_i \quad i \in [0, q]$
- ▶ the PSD of an MA is given by the expression:

$$S(f) = \left| \sum_{l=0}^q b_l \exp(-j2\pi l f / f_e) \right|^2$$

MA models

- ▶ the estimation of the parameters can be done in different ways; we will not go into the details
- ▶ **1.** Method of moments; we solve a non-linear system of this type (we can find a local minimum):

$$\begin{bmatrix} b_0 & \dots & b_q \\ b_1 & \ddots & 0 \\ \vdots & \ddots & \vdots \\ b_q & \dots & 0 \end{bmatrix} \begin{bmatrix} b_0 \\ \vdots \\ b_q \end{bmatrix} = \begin{bmatrix} \hat{R}_{xx}(0) \\ \vdots \\ \hat{R}_{xx}(q) \end{bmatrix}$$

MA models

- ▶ **2.** Long AR method; we start by modeling the signal by a very large AR model ($P > q$); we get the parameters β_j ; we compute their correlation: $\pi_l = \sum_{n=0}^{P-|l|} \beta_n \beta_{n+|l|}$; and we solve a system of this kind:

$$\begin{bmatrix} \pi_0 & \dots & \pi_{q-1} \\ \pi_1 & \ddots & \pi_q \\ \vdots & \ddots & \vdots \\ \pi_{q-1} & \dots & \pi_0 \end{bmatrix} \begin{bmatrix} b_1 \\ \vdots \\ b_q \end{bmatrix} = - \begin{bmatrix} \pi_1 \\ \vdots \\ \pi_q \end{bmatrix}$$

- ▶ **3.** Maximum likelihood method: not widely used, due to the large number of local optima

ARMA models

- ▶ The estimation of the parameters a_k and b_l can be done in different ways; we will not go into the details:
- ▶ 1. estimation of AR parameters (by rewriting the Yule-Walker equations); then MAization of the signal by filtering; and estimation of the MA part
- ▶ 2. long AR method, of order $R = p + q$; the MA part and the AR part separate afterwards; notice that the problem with these “Long AR” methods is that the greater the order, the more poorly estimated the last coefficients.
- ▶ 3. iterative method of Durbin; it goes through a long AR, and by the Durbin method set up for the MAs above
- ▶ 4. least square extension of method 2., taking $R \geq p + q$.

Plan

- ▶ Part 1: Introduction
- ▶ Part 2: More about standard (non-parametric) spectral analysis (periodogram...)
- ▶ Part 3: Parametric spectral analysis: AR/MA/ARMA
- ▶ **Part 4: Advanced spectral analysis: MUSIC, Pisarenko, Prony**
- ▶ Part 5: Laplace, Z, Bode...
- ▶ Part 6: Filtering: analog versus digital
- ▶ Part 7: Advanced spectral analysis: Wavelets
- ▶ Part 8: Matching/basis pursuit
- ▶ Part 9: Sparsity
- ▶ Part 10: Random signals: temporal/spectral properties

Part 4: Advanced spectral analysis: MUSIC, Pisarenko, Prony

Stéphane Rossignol

September 14, 2026

Pisarenko (1973)

- ▶ we no longer consider the same signal model as for AR/MA/ARMA models
- ▶ we assume that the signal is the sum of m sines, mixed with Gaussian white noise:

$$x(n) = \sum_{i=1}^m a_i \cos \left(2\pi f_i \frac{n}{f_e} + \phi_i \right) + n(n)$$

- ▶ we want to estimate the amplitudes a_i , the frequencies f_i and the variance of the noise σ^2
- ▶ note: we drop the phases ϕ_i (anyway, this is consistent with our perspective of sound processing, since our ear is not phase sensitive)

Pisarenko

- **point 1.** the theoretical autocorrelation coefficients of such a signal are (remember the signal studied in the first year):

$$R_{xx}(k) = \frac{1}{2} \sum_{i=1}^m a_i^2 \cos\left(2\pi f_i \frac{k}{f_e}\right) + \sigma^2 \delta(k)$$

- this allows us to go back to the amplitudes of the sines once we have estimated their frequencies (see point 2. below)
- we have indeed (P4):

$$\begin{bmatrix} \cos(2\pi f_1/f_e) & \dots & \cos(2\pi f_m/f_e) \\ \vdots & \ddots & \vdots \\ \cos(2\pi m f_1/f_e) & \dots & \cos(2\pi m f_m/f_e) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_m \end{bmatrix} = \begin{bmatrix} \hat{R}_{xx}(1) \\ \vdots \\ \hat{R}_{xx}(m) \end{bmatrix}$$

Pisarenko

- ▶ we must now invert the matrix of the cos!
- ▶ note: we forget without trouble the noise (intervening only for $k = 0$)
- ▶ from m estimated autocorrelation coefficients and the f_m , after having inverted the matrix of the cos, we easily get the α s
- ▶ and of course the amplitudes of the sines are then:

$$a_i = \sqrt{|2\alpha_i|}$$

- ▶ note: the phases disappear as soon as we limit ourselves to the autocorrelation coefficients

Pisarenko

- ▶ **point 2.** it is a question of finding the frequencies f_m
- ▶ we must now notice that, in the simple case where there is only one sine of frequency f , we have:

$$x(n) = 2 \cos(2\pi f / f_e) x(n-1) - x(n-2)$$

- ▶ note: just remember that:

$$\cos(a) \cos(b) = \frac{1}{2} (\cos(a-b) + \cos(a+b))$$

- ▶ this result can be generalized for a sum of m sines, and we can write:

$$x(n) = \sum_{i=1}^{2m} a_i x(n-i)$$

Pisarenko

- ▶ moreover, it can be generalized to the noisy version of Pisarenko's model; we get then:

$$x(n) - n(n) = \sum_{i=1}^{2m} a_i (x(n-i) - n(n-i))$$

or

$$\sum_{i=0}^{2m} a_i x(n-i) = \sum_{i=0}^{2m} a_i n(n-i) \quad (\text{with } a_0 = 1) \quad \text{P1}$$

- ▶ this is an ARMA signal, but we don't care for the rest: as the AR part and the MA part are identical, the technique used to find the parameters is completely different of those used for ARMA's (good: we do not fall back into auto-regression!)

Pisarenko

- ▶ we need to remember some elements of linear algebra:
 - ▶ since the signal is real, the roots of $A(z)$ (see above) go in conjugate pairs
 - ▶ as the signal is a sum of sines, the roots of this polynomial are on the unit circle
 - ▶ and a polynomial whose roots go in conjugate pairs of modulus 1 is symmetrical
- ▶ we get then: $a_i = a_{2m-i} \quad i \in [0, m]$

Pisarenko

- ▶ we can write **P1** in matrix form: $\mathbf{x}^T \mathbf{a} = \mathbf{n}^T \mathbf{a}$
- ▶ so we also have: $\mathbf{x}\mathbf{x}^T \mathbf{a} = \mathbf{x}\mathbf{n}^T \mathbf{a}$
- ▶ yet the autocorrelation matrix is the expectation of $\mathbf{x}\mathbf{x}^T$:
 $\hat{R}_{xx} = E [\mathbf{x}\mathbf{x}^T]$, who is therefore of size $(2m + 1) \times (2m + 1)$
- ▶ moreover, as the noise is supposed to be uncorrelated to the sum of sines (noted $\mathbf{s}$ here), we get:

$$E [\mathbf{x}\mathbf{n}^T] = E [(\mathbf{s} + \mathbf{n})\mathbf{n}^T] = E [\mathbf{n}\mathbf{n}^T] = \sigma^2 \mathbf{Id}$$

- ▶ which leads us to: $\hat{R}_{xx} \mathbf{a} = \sigma^2 \mathbf{a}$
- ▶ so the vector of parameters $\mathbf{a}$ is the eigenvector of $\hat{R}_{xx}$ associated with the eigenvalue σ^2 (still some algebra)

Pisarenko

- ▶ we must see that this eigenvalue is in fact the smallest eigenvalue of $\hat{R}_{xx}$
- ▶ indeed, we can see that: $\hat{R}_{xx} = R_{ss} + R_{nn}$, or $R_{ss} = \hat{R}_{xx} - \sigma^2 \mathbf{Id}$
- ▶ yet, if we note λ_i the other eigenvalues of $\hat{R}_{xx}$, the eigenvalues of R_{ss} are equal to $\lambda_i - \sigma^2$
- ▶ and as (another linear algebra reminder) R_{ss} is positive definite, since it is an autocorrelation matrix, then all its eigenvalues are positive; we get then: $\sigma^2 < \lambda_i \quad \forall i$, and the demonstration is complete

Pisarenko

- ▶ **point 3.** we are not looking for all the eigenvalues of $\hat{R}_{xx}$
- ▶ there is a recursive algorithm which allows to find the smallest eigenvalue, as well as its associated eigenvector
- ▶ this is the inverse power algorithm; at iteration k , we get:

$$\mu_k = \frac{\mathbf{a}_k^T \hat{R}_{xx} \mathbf{a}_k}{\mathbf{a}_k^T \mathbf{a}_k} \quad \text{and} \quad \mathbf{a}_{k+1} = \hat{R}_{xx}^{-1} \mu_k \mathbf{a}_k \quad \text{P2 and P3}$$

- ▶ the serie μ_k converge towards the smallest eigenvalue of $\hat{R}_{xx}$ and the serie of vectors $\mathbf{a}_k$ converge towards the associated eigenvector

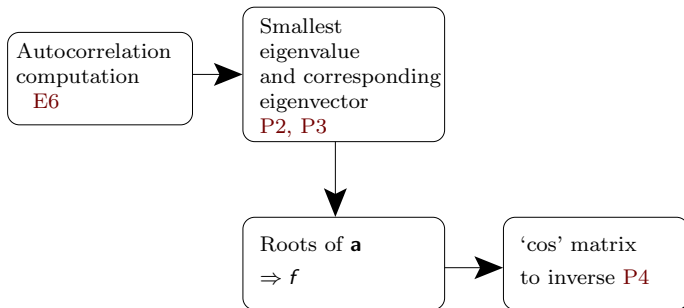
Pisarenko

- ▶ note 1: we initialize μ_0 to 1 and $\mathbf{a}_0$ randomly
- ▶ note 2: moreover, we need a stop criterion for this inverse power algorithm:
 - ▶ when the difference between μ_{k+1} and μ_k is smaller than a certain threshold ($1e^{-10}$ in our case), we decide that it has converged
- ▶ the roots of the polynomial associated with $\mathbf{a}$ are the $\exp(\pm j2\pi f_i/f_e)$ which allow us to find the f_i
- ▶ and we have everything we need

Pisarenko – Companion calculation

- ▶ **be careful:** the initial (historical) method allows to get a spectrum of lines, not a complete PSD; we have:
 - ▶ or a spectrum of lines (we put ϵ to all other frequencies)
 - ▶ or a PSD, by doing as for MUSIC (see later)

Pisarenko – Summary



```

function [ff, mydsp] = mypisarenko(xx, pp, fe)
MM1 = 2*pp; MM=MM1+1; NN=length(xx); xx = xx-mean(xx); %% initializations
acf = xcorr(xx(1:NN), MM1, 'biased'); IMM=length(acf); %% correlation
rrr1 = acf(MM1+1:IMM)';
for ii=1:MM1    rrr1 = [rrr1 acf(MM1+1-ii:IMM-ii)'];    end; %% correlation matrix
rrr1 = rrr1'; rrr = rrr1; irrr=inv(rrr);
mu=1; %% recursion to find the smallest eigenvalue
muold=mu;
aaa=rand(1,2*pp+1);
kk=1;
while kk==1
    mu = aaa*rrr*aaa'/(aaa*aaa');
    aaa = irrr*mu*aaa'; aaa = aaa';
    if (abs(muold-mu)<1e-10)    kk=0;    end; %% stopping criterion
    muold=mu;
end;
racines = roots(aaa); frequ = log(racines)/j/2/pi*fe; frequ = abs(frequ(1:2:end)); %% roots extract. => lines
df=0.9765625; ff=-fe/2:df:fe/2; mydsp=zeros(length(ff),1)+1e-10; %% the DSP is computed every df Hz
for ii=1:pp    for jj=1:pp    coscos(ii,jj) = cos(2.*pi*ii*frequ(jj)/fe);    end;    end;
alpha = inv(coscos)*acf(2*pp+1:3*pp)'; amp = sqrt(2*abs(alpha)); %%% magnitudes
for ii=1:pp %%% power spectral density (it is a line spectrum)
    [mini, posi] = min(abs(ff-frequ(ii))); mydsp(posi) = amp(ii); %% positive frequencies
    [mini, posi] = min(abs(ff+frequ(ii))); mydsp(posi) = amp(ii); %% negative frequencies
end;

```

Summary of the results

	Lev.-Durb.	Burg	Marple	Pisarenko	FFT
1 sine	$p \geq 4$	$p \geq 2$	$3 \leq p \leq 10$ $\sigma^2 = 1e^{-4}$ $4 \leq p \leq 40$ $\sigma^2 = 1e^{-3}$	$p \geq 1$	yes
2 sines	$p = 250$: 63 % $p = 265$: 88 % $p = 300$: 100 %	$p = 4$: 99.68 % $p = 5$: 100 % $p = 6$: 97 % $p = 7$: 7 %	$p = 90$: 16 % $p = 100$: 83 % $p = 110$: 93 % $p = 120$: 86 %	other meth. $p = 40$: 33 % $p = 80$: 60 % $p = 100$: 77 % $p = 150$: 94 %	Black. 32 % Hann. 40 % (Rect. 64 % other meth.)
env.	$n1 : e > 0.318$ $n2 : e > 0.305$	not quite effective	$n1 : e > 0.337$ $n2 : e > 0.310$	$n3 : e > 0.392$	detection peaks to do
20 partials	yes ($p = 200$)	yes ($p = 100$)	yes ($p = 140$)	difficult	yes
pink noise	$n1 : e > 0.568$ $n2 : e > 0.367$ $n3 : e > 0.378$	$n1 : e > 0.708$ $n2 : e > 0.456$ $n3 : e > 0.493$	$n1 : e > 0.520$ $n2 : e > 0.344$	$n1 : e > 0.99$	$e \simeq 0.5$

MUSIC

- ▶ MUSIC (MULTiple Signal Characterization) is a direct extension of Pisarenko
- ▶ in regards to MUSIC, instead of considering only the smallest eigenvalue of autocorrelation matrix of size M , we consider the $M - p$ smallest
- ▶ we then decompose this autocorrelation matrix into two subspaces, which are orthogonal (in can be proven, in linear algebra):
 - ▶ a signal subspace of dimension p , composed of the eigenvectors $\mathbf{w}_k$ corresponding to the greatest eigenvalues
 - ▶ and a noise subspace of dimension $M - p$, composed of the eigenvectors $\mathbf{v}_k$ corresponding to the smallest eigenvalues (supposed to be equal to σ^2)

MUSIC

- ▶ and instead of projecting only onto the eigenvector corresponding to the smallest eigenvalue, we project onto the noise subspace (which we could have done instead of looking for the roots of $\mathbf{a}$ for Pisarenko: see above)
- ▶ like for Pisarenko, then, we get: $\hat{R}_{xx} = R_{ss} + R_{nn}$, or also $R_{ss} = \hat{R}_{xx} - \sigma^2 \mathbf{Id}$
- ▶ considering Pisarenko, the matrices were of dimension $(2m + 1) \times (2m + 1)$ (m being the number of real sines/lines considered)
- ▶ for MUSIC, they are of dimension $M \times M$, with M a priori much bigger than $2m + 1$

MUSIC

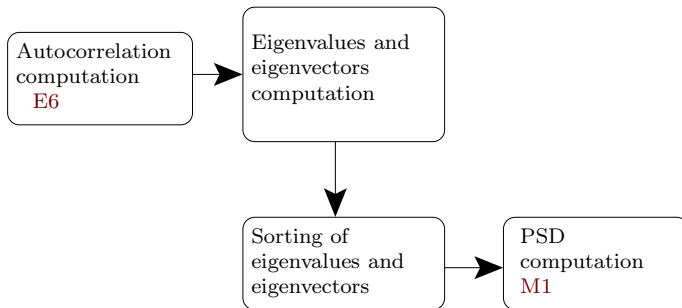
- ▶ the PSD is then equal to:

$$P(f) = \frac{1}{\mathbf{e}^H(f) \left[\sum_{k=p+1}^M \mathbf{v}_k \mathbf{v}_k^H \right] \mathbf{e}(f)} \quad \text{M1}$$

with: $\mathbf{e}(f) = \left(1, \cos\left(2\pi \frac{f}{f_e}\right), \dots, \cos\left(2\pi \frac{(M-1)f}{f_e}\right) \right)$ (if the lines were complex, we would have exponentials)

- ▶ note: in the same way that we could use this to obtain a PSD with Pisarenko instead of a “spectrum of lines”, we could for MUSIC extract lines instead of a real PSD; this would be the method called root-MUSIC

MUSIC – Summary



MUSIC – Matlab/octave code

```
function [ff, mydsp] = mymusic(xx, pp, MM, fe)
if (MM<=pp)    fprintf(1, 'It is absolutely essential that MM>pp !!!');    break;    end;
MM1=MM-1;xx = xx-mean(xx);
acf = xcorr(xx, MM1, 'unbiased'); %%% correlation
lmm=length(acf);
rrr = acf(MM1+1:lmm)';
for ii=1:MM1
    rrr = [rrr acf(MM1+1-ii:lmm-ii)'];
end;
rrr = rrr';
[v, lambda] = eig(rrr); %%% direct method for finding all eigenvalues
lamb = diag(lambda);
[vl,pl] = sort(lamb,'descend');
df=0.9765625; %%% power spectral density – the PSD is computed every df Hz
ff=-fe/2:df:fe/2;
mydsp=zeros(length(ff),1);
deni=zeros(MM,MM);
for ii=pp+1:MM    deni = deni + v(:,pl(ii))*conj(v(:,pl(ii)))';    end;
for ii=1:length(ff)
    ee = cos(2*pi*ff(ii)*[0:MM1]/fe);
    den = conj(ee)*deni*ee';
    mydsp(ii) = abs(1/den);
end;
mydsp = mydsp-min(mydsp); %%% we optionally remove a non-zero component
```

Summary of the results

	Lev.-Durb.	Burg	Marple	Pisarenko	MUSIC	FFT
1 sin	$p \geq 4$	$p \geq 2$	$3 \leq p \leq 10$ $\sigma^2 = 1e^{-4}$ $4 \leq p \leq 40$ $\sigma^2 = 1e^{-3}$	$p \geq 1$	$p = 1 \Rightarrow$ $M \geq 110$ $p \geq 2 \Rightarrow$ $M \geq p + 2$	yes
2 sin	$p=250$ 63% $p=265$ 88% $p=300$ 100%	$p=4$ 99.68% $p=5$ 100% $p=6$ 97% $p=7$ 7%	$p=90$ 16% $p=100$ 83% $p=110$ 93% $p=120$ 86%	other meth. $p=40$ 33% $p=80$ 60% $p=100$ 77% $p=150$ 94%	$p=4 M=90$ 23% $p=4 M=100$ 73% $p=4 M=110$ 98.6% $p=4 M=120$ 68%	Black. 32% Hann. 40% <i>Rect. 64%</i> <i>other meth.</i>
env	$e_1 > 0.318$ $\bar{e}_2 > 0.305$	not very effective	$e_1 > 0.337$ $\bar{e}_2 > 0.310$	$e_3 > 0.392$	$e_1 > 0.527$ $\bar{e}_2 > 0.462$	detection of peaks to do
20 sin	yes ($p=200$)	yes ($p=100$)	yes ($p=140$)	difficult	yes ($p=50 M=150$)	yes
pink	$\bar{e}_1 > 0.568$ $\bar{e}_2 > 0.367$ $e_3 > 0.378$	$\bar{e}_1 > 0.708$ $\bar{e}_2 > 0.456$ $e_3 > 0.493$	$\bar{e}_1 > 0.520$ $\bar{e}_2 > 0.344$	$\bar{e}_1 > 0.99$	$\bar{e}_1 > 0.746$ $\bar{e}_2 > 0.527$	$e \simeq 0.5$

Prony

- ▶ Prony's method is often considered to be another possible extension of Pisarenko
- ▶ even if it was invented before that of Pisarenko: indeed, Prony's method dates from 1795
- ▶ we will only examine the initial (historical) Prony method: the extended method (also called Least Squares Prony) is largely a rewritten version of the AR model

Prony

- ▶ on the other hand, the initial (historical) method consists in estimating p complex lines from $2p$ signal samples only
- ▶ that is to say, the signal length and the order of the model are related
- ▶ however, we will see that even this we will continue to consider signals of 1280 samples, as before for the tests
- ▶ of course, if the signal is real, from our $2p$ samples, we get $p/2$ sines in the positive frequencies and their $p/2$ counterparts in negative frequencies

Prony

- ▶ the signal model is again different; we use:

$$x_n = \sum_{i=1}^p h_i z_i^n \text{ for } n \geq 0$$

where the h_i are complex amplitudes: $h_i = \rho_i \exp(j\Theta_i)$; and where the z_i are complex exponentials, modeling damped sines: $z_i = \exp(\alpha_i + j2\pi f_i/f_e)$

- ▶ we prove that the signal follows a regression relation of the type of that of the AR model (except that the input is not noise)

- ▶ **point 1.** indeed, we define the polynomial $P(z) = \prod_{i=1}^p (z - z_i)$,

where the z_i are the exponentials seen above

Prony

- ▶ then we can expand it to get: $P(z) = \sum_{i=0}^p a_i z^{p-i}$

- ▶ yet, we have: $\sum_{i=0}^p a_i x_{k-i} = \sum_{i=0}^p a_i \left(\sum_{j=1}^p h_j z_j^{k-i} \right)$

- ▶ we can invert the two sums and we get:

$$\sum_{j=1}^p h_j \left(\sum_{i=0}^p a_i z_j^{k-i} \right) = \sum_{j=1}^p h_j z_j^{k-p} \left(\sum_{i=0}^p a_i z_j^{p-i} \right)$$

- ▶ the term in parenthesis is equal to zero by definition of the polynomial $P(z)$

Prony

- therefore, we have: $\sum_{i=0}^p a_i x_{k-i} = 0$, that is to say also:

$$a_0 x_k = - \sum_{i=1}^p a_i x_{k-i}$$

- a_0 is equal to 1: just inspect $P(z)$ to be convinced
- then, from $2p$ samples (on the N that we have at our disposal, remember), we can write:

$$\begin{bmatrix} x_{p-1} & x_{p-2} & \dots & x_0 \\ x_p & x_{p-1} & \dots & x_1 \\ \vdots & \vdots & \dots & \vdots \\ x_{2p-2} & x_{2p-3} & \dots & x_{p-1} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = - \begin{bmatrix} x_p \\ x_{p+1} \\ \vdots \\ x_{2p-1} \end{bmatrix} \quad Y1$$

Prony

- ▶ we invert this matrix, one way or another, to obtain the vector **a**
- ▶ we can then calculate the roots of the polynomial $P(z)$, in order to estimate the $\hat{z}_i$
- ▶ then, we obtain the estimates of the damping coefficients and of the frequencies of the modes like this:

$$\hat{\alpha}_i = \log |z_i| \text{ and } \hat{f}_i = \frac{1}{2\pi} \arctan \left(\frac{\text{Im}(z_i)}{\text{Re}(z_i)} \right)$$

- ▶ **point 2.** it remains to estimate the complex amplitudes
- ▶ to do this, we start again from the definition of the model

Prony

- ▶ indeed, we can now form another linear system:

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ z_1 & z_2 & \dots & z_p \\ \vdots & \vdots & \dots & \vdots \\ z_1^{p-1} & z_2^{p-1} & \dots & z_p^{p-1} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_p \end{bmatrix} = - \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{p-1} \end{bmatrix} \quad Y_2$$

- ▶ we invert this matrix, one way or another, to obtain the vector **h**
- ▶ and we obtain the estimates of the amplitudes and of the phases as follows:

$$\hat{\rho}_i = |h_i| \text{ and } \hat{\Theta}_i = \arctan \left(\frac{\text{Im}(h_i)}{\text{Re}(h_i)} \right)$$

Prony

- ▶ **point 3.** it still remains to calculate the spectrum of Prony
- ▶ we can be satisfied with the line spectrum, since we collect frequencies and amplitudes (we get the same type of spectra as with the Pisarenko method)
- ▶ or we can calculate a PSD; to improve the resolution, we symmetrize the signal x_n this way:

$$x_n = \sum_{i=1}^p h_i z_i^n \quad \text{for } n \geq 0$$
$$x_n = \sum_{i=1}^p h_i (z_i^*)^n \quad \text{for } n < 0$$

Prony

- ▶ the Z transform of this signal is:

$$X(z) = \sum_{i=1}^p h_i \left(\frac{1}{1 - z_i z^{-1}} - \frac{1}{1 - (z_i^* z)^{-1}} \right)$$

- ▶ and, as usual, the resulting PSD is:

$$S(f) = \left| \frac{1}{f_e} X(\exp(j2\pi f / f_e)) \right|^2 \quad \text{Y3}$$

- ▶ **point 4.** and finally it remains to solve the cases of bad conditioning for the two matrices to be inverted

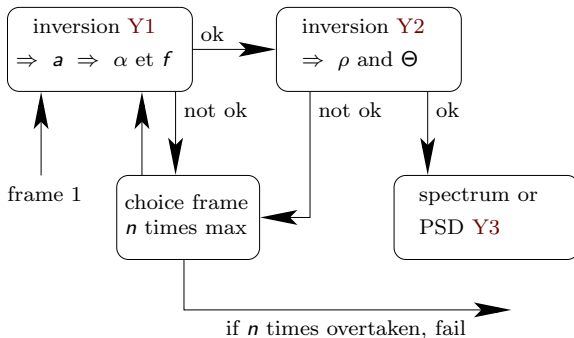
Prony

- ▶ in fact, the matrices to invert (see **Y1** and **Y2**) are not necessarily well conditioned, and therefore are not easily invertible
- ▶ thus, when this arrive, we take another part of the signal than the $2p$ first samples on the $N = 1280$ available; and we work on this new short frame
- ▶ this is what has been implemented in *myprony_matlab.m*
- ▶ in the event of poor conditioning of one of the 2 matrices, we randomly choose another signal short frame, among the 1280 samples available

Prony

- ▶ this operation is repeated, until the 2 matrices have been inverted, or until this operation have been done a given number of times
- ▶ this operation therefore introduces a new parameter in the model
- ▶ we simply called it n
- ▶ note: often, it's **Y2** which is problematic, hence a waste of time, as the inversion of **Y1** has been already done, unnecessarily

Prony – Summary



Prony – Matlab/octave code – 1/3

```
function [aa, ff, mydsp] = myprony(xx, pp, nco, fe, rec)
tsig = length(xx);
df=0.9765625; %%% the PSD is computed every df Hz
ff=-fe/2:df:fe/2;
strt=0; %%% starting sample
count=0;
done=0;
while (done<1) %%% start of main loop
    ok1=0;
    ok2=0;
    %%% alpha and f
    yy=[];
    for (ii=1:pp)
        yy = [yy xx(pp+ii-1+strt:-1:ii+strt)'];
    end;
    yy=yy';
    y1 = xx(pp+1+strt:2*pp+strt)';
    [invaa,condit]=inv(yy);
    if (condit>1.52721e-17) %%% condition number Y1
        ok1=1;
        aa = -invaa*y1;
        aa = [1 aa]';
        racines=roots(aa);
        alpha = log10(abs(racines));
        frequ = 1/2/pi*atan2(imag(racines),real(racines));
```

Prony – Matlab/octave code – 2/3

```
%%%% rho and theta
zz=[];
for (ii=0:pp-1)
    zz = [zz racines.^ii'];
end;
zz=zz';
y2 = xx(1:pp)';
[invzz,condit]=inv(zz);
if (condit>1.52721e-17) %%%% condition number Y2
    ok2=1;
end;
end;
if (ok1==1 & ok2==1)
    hh = invzz*y2;
    rho = abs(hh);
    theta = atan2(imag(hh),real(hh));
    done=1;
else
    fprintf(1,'pc ');fflush(1);
    count=count+1;
    if (count==nco)
        done=2;
    end;
    strt=round(rand(1,1)*(tsig-2*pp)); %%%% choosing a new starting point
end;
```

Prony – Matlab/octave code – 3/3

```

end; %%%% end of main loop
ff=ff/fe; %%%% power spectral density
if (done==1)
    if (rec==0)
        %%%% PSD Prony
        sff=zeros(1,length(ff));za=exp(-j*2*pi*ff);zb=conj(za)
        for ii=1:pp
            sff = sff + hh(ii) * ( 1./(1 - racines(ii)*za ) - 1./(1 - ( conj(racines(ii))*zb ).^(-1) ) );
        end;
        mydsp=abs(sff).^2/fe;
    else
        %%%% line spectrum
        mydsp=zeros(length(ff),1)+1e-10;
        for ii=1:pp
            [mini, posi] = min(abs(ff-frequ(ii)));mydsp(posi) = abs(hh(ii));
            [mini, posi] = min(abs(ff+frequ(ii)));mydsp(posi) = abs(hh(ii));
        end;
    end;
else
    fprintf(1,'unable to comply ');fflush(1);
    mydsp=zeros(1,length(ff))+1e-15;
    mydsp(round(length(ff)/2))=1e-5;
    aa=[];
end;
ff=ff*fe;

```

Summary of the results

	L.-Durb.	Burg	Marple	Pisarenko	MUSIC	Prony-H	FFT
1 sin	$p \geq 4$	$p \geq 2$	$3 \leq p \leq 10$ $\sigma^2 = 1e^{-4}$ $4 \leq p \leq 40$ $\sigma^2 = 1e^{-3}$	$p \geq 1$	$p = 1 \Rightarrow$ $M \geq 110$ $p \geq 2 \Rightarrow$ $M \geq p + 2$	$p \geq 40$ $n \geq 10$	yes
2 sin	$p=250$ 63% $p=265$ 88% $p=300$ 100%	$p=4$ 99.68% $p=5$ 100% $p=6$ 97% $p=7$ 7%	$p=90$ 16% $p=100$ 83% $p=110$ 93% $p=120$ 86%	other way $p=40$ 33% $p=80$ 60% $p=100$ 77% $p=150$ 94%	$p = 4$ $M_{=90}$ 23% $M_{=100}$ 73% $M_{=110}$ 98.6% $M_{=120}$ 68%	other way $p \geq 180$ $n \geq 10$ $\leq 87\%$	B 32% Hn 40% R 64% other meth.
env	$e_1 : 0.318$ $\bar{e}_2 : 0.305$	not very effective	$e_1 : 0.337$ $\bar{e}_2 : 0.310$	$e_3 : 0.392$	$e_1 : 0.527$ $\bar{e}_2 : 0.462$	$e_1 : 0.043$ $\bar{e}_2 : 0.044$	det. peak to do
20 sin	yes $p=200$	yes $p=100$	yes $p=140$	other	yes $pM_{=50,150}$	yes $pn_{=150,10}$	yes
pink	$\bar{e}_1 : 0.568$ $\bar{e}_2 : 0.367$ $e_3 : 0.378$	$\bar{e}_1 : 0.708$ $\bar{e}_2 : 0.456$ $e_3 : 0.493$	$\bar{e}_1 : 0.520$ $\bar{e}_2 : 0.344$	$\bar{e}_1 : 0.99$	$\bar{e}_1 : 0.746$ $\bar{e}_2 : 0.527$	$\bar{e}_1 : 0.690$ $\bar{e}_2 : 0.608$	$e \simeq 0.5$

Other performances of these methods

- ▶ there is a whole bunch of other tests that should be done
- ▶ we will only consider a very small part of these: what concerns the robustness of spectral analysis methods to noise
- ▶ we will take the signal composed of a single sine, to which we add noise of variable variance
- ▶ note: we come across problems where there are several parameters which vary; here, it is p that needs to be optimized according to σ^2
- ▶ we will therefore not be able to be exhaustive, neither near nor far

Other performances of these methods

- ▶ Levinson-Durbin, Burg and Marple behave at least as well as the FFT
- ▶ the made measures do not allow them to be classified more finely
 - ▶ for instance, at SNR=-20 dB, we are at best around 30 % of good detections (GD) for all spectral analysis methods
 - ▶ the transition to more than 75 % of GD takes place between $\sigma^2 = 30$ and $\sigma^2 = 15$
- ▶ Pisarenko seems less robust than the 3 AR methods: at SNR=-20 dB, we have 15 % of GD, and the transition to more than 75 % GD is between $\sigma^2 = 15$ and $\sigma^2 = 7.5$; fine tuning the order doesn't seem to help

Other performances of these methods

- ▶ MUSIC seems less robust than the 3 AR methods and behaves almost the same way as Pisarenko: at SNR=-20 dB, we have about 15 % of GD, and the transition to more than 75 % of GB is between $\sigma^2 = 15$ and $\sigma^2 = 7.5$; the fine tuning of M is to be done more judiciously according to the order
- ▶ Prony is much more sensitive to noise than the other methods; even with an SNR of 17 dB, we still have less than 75 % of good detections; increasing the order doesn't seem to help (fine tuning of n ?)

Conclusion

- ▶ we have shown that the so-called “high-resolution” methods are indeed and efficiently high-resolution methods
- ▶ we have shown that we can easily use them either to extract spectra of lines or to extract spectral envelopes (PSD)
- ▶ the experimental rather than theoretical bias of the course aims to show in a concrete way the difficulties encountered during the qualification of methods of spectral analysis (among others): only statistical tests, starting here (for spectral analysis) of perfectly calibrated signals, can give an idea of the performance of a particular tool